

Reviewer Pack — Free Entropy Gap in Read-Twice BP Transition Matrices

Entry 961d26b09f3e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 20:44:51 UTC

Free Entropy Gap in Read-Twice BP Transition Matrices

Entry ID: 961d26b09f3e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 20:44:51 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice BP P with m states, let $\rho(P)$ be the free entropy of its transition matrix (computed via the R-transform). Then $\rho(P) = O(\log m)$ for all P , but $\rho(\text{IP}_2) = \Omega(n)$ where IP_2 is the inner product mod 2 BP requiring exponential size.

Rationale (proposer’s reasoning):

Free entropy quantifies non-commutative structure in transition matrices. Read-once BPs have low non-commutative complexity, while read-twice BPs may exhibit higher free entropy. The IP_2 BP’s exponential size suggests a phase transition in free entropy thresholds.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e9b4f5e674f0be50

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(n):
                if j != i:
                    factor = A[j][i] / A[i][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def r_transform(A):
        m, n = len(A), len(A[0])
        if m != n:
            raise ValueError("Matrix must be square")

        # Perform Gaussian elimination to get the upper triangular form
        U = gaussian_elimination(A)

        # Calculate the R-transform
        log_det = 0.0
        for i in range(n):
            log_det += math.log(abs(U[i][i]))

        return log_det
```

```

def generate_random_matrix(m, n):
    return [[random.choice([-1, 1]) for _ in range(n)] for _ in range(m)]

def is_inner_product_mod_2_BP(A):
    m, n = len(A), len(A[0])
    if m != n:
        return False
    for i in range(m):
        for j in range(i + 1, m):
            if A[i][j] != -A[j][i]:
                return False
    return True

n = 40
P = generate_random_matrix(n, n)
IP_2 = generate_random_matrix(n, n)

if not is_inner_product_mod_2_BP(IP_2):
    return {
        "metric_name": "free_entropy",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

rho_P = r_transform(P)
rho_IP_2 = r_transform(IP_2)

return {
    "metric_name": "free_entropy",
    "metric_value": max(rho_P, rho_IP_2),
    "instances_tested": 1,
    "conjecture_holds": rho_IP_2 >= 10 * n and rho_P <= 10 * math.log(n),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{first_failing_seed}' first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_holds': Fals
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
TRIAL: {'metric_name': 'free_entropy', 'metric_value': None, 'instances_tested': 1, 'conjecture_hold
RESULT: FALSIFIED counterexample='' first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

n=1 instances_tested violates the $n \leq 15$ threshold, making scaling analysis invalid. The 'mapping_undefined' error suggests metric definition bugs or construction gaps, as free entropy computation via R-transform requires proper matrix mappings.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test failed to meet the support fraction threshold (0% support) and the counterexample 'mapping_undefined' suggests metric definition bugs. Insufficient instances tested (n=1) invalidates scaling analysis. | next: Fix metric definition for free entropy computation and retest with ≥ 15 instances

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	47490
2	preregistration	ollama_remote	qwen3:8b	0	0	31621
3	novelty	ollama_remote	qwen3:8b	0	0	27343
4	novelty	ollama_remote	qwen3:8b	0	0	22915
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12812
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10337
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10075

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11722
9	critic	ollama_remote	qwen3:8b	0	0	38618
10	judge	ollama_remote	qwen3:8b	0	0	22270

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 235202 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/961d26b09f3e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/961d26b09f3e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/961d26b09f3e.tar.gz (if generated)